Python for GenAI

Basics of Python

1. **Installations**
   - 1.1. Install Python
   - 1.2. Install VSCode
   - 1.3. Install Anaconda/Miniconda
   - 1.4. Create a conda environment
   - 1.5. Create a virtual environment using venv
2. **Simple Calculations**
   - 2.1. Write a program to add, subtract, multiply, and divide two numbers.
   - 2.2. Write a program to convert temperatures from Celsius to Fahrenheit.
   - 2.3. Create a program that calculates the area of a circle given its radius.
   - 2.4. Write a program to swap the values of two variables.
3. **Control Flow Basics**
   - 3.1. Write a program that asks the user for their age and determines if they are eligible to vote (18 years or older).
   - 3.2. Create a simple program to check if a given number is even or odd.
   - 3.3. Write a program that asks the user to input three numbers and prints the largest.
   - 3.4. Develop a program that checks whether a given year is a leap year.
4. **List Operations**
   - 4.1. Write a program that takes a list of numbers and outputs the sum and average.
   - 4.2. Create a program to find the maximum and minimum elements in a list.
   - 4.3. Write a program that removes duplicates from a list and prints the unique elements.
   - 4.4. Develop a program that counts the number of occurrences of each element in a list.
5. **Loops and Iteration**
   - 5.1. Write a program that prints the first 10 Fibonacci numbers.
   - 5.2. Create a program that counts the number of vowels in a given string.
   - 5.3. Develop a program that prints a multiplication table for numbers 1 to 10.
   - 5.4. Write a program that finds all prime numbers between 1 and 100.
6. **Functions**
   - 6.1. Write a function to find the factorial of a number.
   - 6.2. Create a function that takes a list of numbers and returns the list sorted in ascending order.
   - 6.3. Develop a function that checks whether a given string is a palindrome.
   - 6.4. Write a function that takes two lists and returns a list containing only the common elements.

7. **Error Handling**
    7.1. Write a program that asks the user for a number and handles the `ValueError` if the input is not a number.
    7.2. Create a function that divides two numbers and handles division by zero.
    7.3. Develop a program that reads a file and handles the `FileNotFoundError` if the file does not exist.
    7.4. Write a program that attempts to open a file, reads its contents, and handles any I/O errors that may occur.
8. **OOP**
    8.1. Create a `BankAccount` class with methods to deposit, withdraw, and check the balance.
    8.2. Implement an `Employee` class with attributes like name, age, and salary, and methods to calculate annual salary and display employee details.
    8.3. Develop a `Rectangle` class that includes methods to calculate the area and perimeter, and create instances to demonstrate their use.
    8.4. Write a `Student` class that stores details such as name, age, and grades, with a method to calculate the average grade.
9. **File Handling**
    9.1. Write a program to read a text file and count the number of words and lines.
    9.2. Create a program that writes user input to a file and then reads the content back to the user.
    9.3. Develop a program that appends data to an existing file and then reads the entire file's content.
    9.4. Write a program that reads a CSV file and converts it into a list of dictionaries, where each dictionary represents a row.
10. **Advanced List Operations**
    10.1. Write a program using list comprehensions to create a list of squares of even numbers from 1 to 20.
    10.2. Create a generator that yields prime numbers up to a specified limit.
    10.3. Develop a program that flattens a nested list using recursion.
    10.4. Write a program that generates all possible combinations of two lists using list comprehensions.
11. **DSA**
    11.1. Write a program to rotate an array of integers to the right by a given number of steps.
    11.2. Implement a binary search algorithm in Python to find the position of a target value within a sorted array.
    11.3. Create a singly linked list in Python and implement methods to add, remove, and search for elements.
    11.4. Implement a stack using Python's list with methods for push, pop, peek, and checking if the stack is empty.
    11.5. Implement a queue using two stacks in Python, with methods for enqueue, dequeue, and checking if the queue is empty.

11.6. Write a program to perform in-order, pre-order, and post-order traversal on a binary tree.
11.7. Implement a graph using an adjacency list in Python, and write methods for adding vertices, edges, and performing BFS (Breadth-First Search).
11.8. Implement the merge sort algorithm in Python to sort an array of integers.
11.9. Solve the Fibonacci sequence problem using dynamic programming to optimize the solution.
11.10. Create a simple hash table in Python with basic operations like insert, delete, and search, handling collisions using chaining.

Resources

Here are some highly recommended online resources to start learning Python:

1. **LearnPython.org** offers a hands-on approach to learning Python, with interactive coding challenges and a sandbox environment for beginners. It's ideal for those who prefer learning by doing

2. **Codecademy's Python Course** provides both free and paid interactive programming courses. It's a structured platform that helps you learn Python from the basics to more advanced concepts through active coding exercises

3. **W3Schools** has a comprehensive free Python tutorial that covers various topics from basics to more advanced concepts such as file handling and using databases. It offers an interactive environment where you can practice coding directly on the website

4. **Real Python** is perfect for those who prefer in-depth articles and video courses. It provides a wide range of tutorials that cover beginner to advanced Python topics, supplemented by engaging visuals and quizzes to test your knowledge

5. **Google's Python Class** is another excellent resource, particularly if you already have some programming experience. This free course covers essential Python programming topics and includes hands-on exercises to help you learn quickly and effectively

6. **Python.org** itself is a great starting point, especially for beginners. It offers a beginner's guide and a comprehensive list of Python libraries and frameworks, along with tutorials for more experienced users

Each of these resources has its unique strengths, so you might find it helpful to explore several to see which ones suit your learning style best.